# Cómo se organiza el proyecto

Una guía sencilla para entender Jugadores, los catálogos de Biblioteca y los helpers. Actualizada el 4 de noviembre de 2026.

## 1. La base del sistema

Se utiliza Laravel para organizar el proyecto y MySQL para guardar la información. Los archivos SQL están en la raíz. Este proyecto **no utiliza migraciones**.

**jugadores.sql** crea la base del sistema y las cuentas de ejemplo. Se importa primero, en una base vacía. **biblioteca.sql** se importa después en esa misma base: agrega los cinco catálogos, sus relaciones, los permisos y los datos recuperados de Biblioteca.

Si Jugadores ya está instalado, solo hace falta importar biblioteca.sql. La conexión se configura en **.env**. Los SQL indican UTF-8 para conservar los acentos y la ñ.

## 2. Cada parte tiene una tarea

| Archivo o carpeta | Para qué sirve |
|---|---|
| routes/web.php | Define las direcciones y los permisos necesarios para abrirlas. |
| app/Http/Controllers | Recibe formularios, valida los datos y guarda los cambios. |
| app/Models | Consulta las tablas y conecta sus relaciones. |
| resources/views | Contiene las pantallas y los formularios. |
| resources/views/layouts/ master.blade.php | Comparte el menú, la barra superior y el espacio del contenido. |
| app/helpers.php | Reúne las funciones para permisos y fotografías. |

## 3. El registro con foto

La persona elige Jugador o Administrador, escribe sus datos y selecciona una foto. LoginController valida el formulario y crea la cuenta y su perfil juntos. Si falla el registro, revierte los datos y elimina la foto que se haya guardado.

La cuenta se guarda en **users**; el alias y la ruta de la foto, en **jugador**. El helper guardar_foto() guarda la imagen dentro de public/fotos/jugador. Se acepta JPG, PNG o WebP de hasta 2 MB.

**Recorrido de los datos**
Formulario > Controlador y validación > Helper de foto y base de datos > Inicio

# Los catálogos de Biblioteca

La gestión que estaba en el proyecto anterior ahora forma parte de Jugadores y utiliza su misma plantilla.

## 4. Qué se puede administrar

| Catálogo / tabla | Qué guarda |
|---|---|
| Autores / autor | Nombre y foto opcional del autor. |
| Categorías / categoria | Nombre de la categoría que organiza las obras. |
| Obras / obra | Nombre, autor, categoría y foto opcional. |
| Partidas / partida | Clave, estado numérico y fecha de creación. |
| Participantes / partidaxusuario | La relación entre una partida y una cuenta existente de users. |

Se conservaron las cuentas de Jugadores para iniciar sesión. Los participantes se eligen desde esas cuentas: no se utiliza un segundo acceso de Biblioteca.

## 5. Cómo trabajan los formularios

En cada catálogo, **Agregar** abre un formulario vacío y **Editar** muestra los datos del registro. Al pulsar **Guardar registro**, el controlador revisa la información antes de guardarla. El botón **Eliminar registro** pide confirmación.

Los métodos de los controladores siguen el mismo recorrido: **index()** muestra el listado, **form()** abre el formulario y **save()** guarda o elimina según el botón elegido.

Si hay errores, aparece un aviso y el formulario conserva lo escrito. Las obras requieren un autor y una categoría existentes. Las fotos de autores y obras son opcionales; se acepta JPG, PNG, GIF o WebP de hasta 2 MB.

## 6. Reglas para cuidar las relaciones

**Autores y categorías:** no se pueden eliminar mientras tengan obras asociadas. Primero se cambian o eliminan esas obras.

**Participantes:** una cuenta puede estar en varias partidas, pero no repetirse dentro de la misma. Al quitar una inscripción se conservan la cuenta y la partida. Al borrar una partida o una cuenta, se eliminan sus inscripciones.

**Partidas:** la fecha de creación se coloca al agregar el registro y se conserva al editarlo.

**Lo recuperado del proyecto anterior**
1 autor, 2 categorías, 2 obras, 1 partida y 3 imágenes. Los registros están en biblioteca.sql y las imágenes están en public/fotos/autores y public/fotos/obras.

# Roles, permisos y helpers

Un helper es una función pequeña que se reutiliza para no escribir la misma tarea en varios lugares.

## 7. Los permisos dependen del rol

Cada usuario tiene un rol y cada rol tiene permisos. La tabla **rolxpermiso** conecta los roles con sus permisos. El Jugador ve normalmente su perfil; el Administrador ve su panel y los catálogos que tenga permitidos.

Ambos roles entran al mismo Inicio. Laravel mantiene la sesión después de comprobar el correo y la contraseña. La contraseña se guarda como una huella de seguridad, no como texto legible. Cerrar sesión termina el acceso.

Para cambiar los accesos, entra como administrador, abre **Roles**, pulsa **Permisos** en el rol elegido, marca o desmarca las opciones y guarda. Los permisos de Biblioteca se asignan inicialmente al rol Administrador al importar su SQL.

## 8. Los tres helpers de app/helpers.php

| Helper | Qué hace y dónde se usa |
|---|---|
| tiene_permiso($clave) | Consulta los permisos del usuario actual. Se usa en el menú y en el inicio; sin sesión devuelve false. |
| guardar_foto($archivo, $carpeta) | Guarda una imagen ya validada con nombre único y devuelve su ruta. Se usa en autores, obras y registro de jugadores. |
| eliminar_foto($foto) | Borra el archivo si existe. Se usa al reemplazar fotos, eliminar registros o deshacer un registro fallido. |

**El helper de permisos reutiliza la lógica existente:** llama al método tienePermiso() del usuario, que revisa los permisos de su rol. No crea otra lista de permisos.

```
@if(tiene_permiso('administrar_obra'))
    <a href="{{ route('obras.index') }}">Obras</a>
@endif
```

Este ejemplo muestra el enlace Obras solo cuando la cuenta tiene ese permiso. Las rutas también están protegidas: **auth** comprueba la sesión y **CandadoPermisosUsuario** comprueba el permiso, aunque se escriba la dirección directamente.

## 9. Validar y guardar son tareas distintas

El controlador comprueba los campos, el tipo de imagen y su tamaño. Después llama al helper para guardarla. Al editar sin subir otra imagen, se conserva la anterior; al reemplazarla, la anterior se borra después de guardar el cambio.

Composer carga app/helpers.php mediante la sección **autoload.files** de composer.json. Si se instala el proyecto o cambia esa configuración, **composer dump-autoload** actualiza la carga de estas funciones.

# Cómo ponerlo a prueba

Un recorrido corto para revisar los catálogos, las fotos y los accesos del rol administrativo.

## 10. Preparación

**Instalación nueva:** configura .env, importa jugadores.sql en una base vacía y después importa biblioteca.sql en esa misma base.

**Instalación existente:** importa únicamente biblioteca.sql para agregar los catálogos. Puede repetirse: los registros recuperados se agregan si su ID no existe y los permisos no se duplican. También corrige el nombre del permiso de categorías si quedó mal codificado en una importación anterior.

Si los estilos no se muestran actualizados, ejecuta **npm run build**. Para trabajar con los archivos de desarrollo, puedes usar **composer run dev**.

**Cuentas de ejemplo incluidas en jugadores.sql**
Administrador: admin@subasart.test
Jugador: jugador@subasart.test
Contraseña de ambas: **password**
Son cuentas para revisar el proyecto en el entorno escolar.

## 11. Prueba paso a paso

**1.** Entra como administrador. Comprueba que aparecen Usuarios, Permisos, Roles, Obras, Autores, Categorías, Partidas y Participantes.

**2.** Agrega un autor y una categoría. Crea una obra usando ambos y sube una foto. Revisa que aparece en el listado.

**3.** Edita la obra sin subir otra imagen: debe conservar su foto. Después cámbiala por otra y comprueba el reemplazo.

**4.** Crea una partida y agrega una cuenta como participante. Intenta inscribirla otra vez: debe aparecer un aviso.

**5.** Intenta eliminar el autor o la categoría que usa la obra. El sistema debe impedirlo mientras sigan asociados.

**6.** En Roles, asigna temporalmente Administrar obras al rol Jugador. Entra con una cuenta de ese rol y comprueba que aparece Obras. Retira el permiso y vuelve a revisar el acceso.

**7.** Cierra sesión. Al intentar abrir un catálogo privado, debe pedirte iniciar sesión.

## 12. Dónde revisar si algo falla

Los mensajes de validación en español están en **lang/es/validation.php**. El diseño compartido está en **resources/views/layouts/master.blade.php** y **resources/css/app.css**. Los comentarios del código explican cada bloque con frases breves.

Las pruebas automáticas están en **tests/Feature/ProyectoTest.php** y **tests/Feature/BibliotecaTest.php**. Se ejecutan con **php artisan test --compact** y utilizan una base temporal para no modificar los registros reales.